Regras de negócio por perfil

Controle de Saída e Chegada de Veículos - AGU

Gerado em 12/11/2026 a partir do código (backend, migrations e frontend), com um agente de análise por perfil. Documento de análise: contém dados de regra, não de servidores (LGPD). Em conflito, vale o CLAUDE.md.

1. Visão geral

O backend manda, o frontend só exibe. O perfil vem de `perfis` no banco, nunca do cliente; botões aparecem conforme `acoes_permitidas` devolvido pela API. Cinco perfis: solicitante, autorizador, admin, motorista e vigilante.

Estados da requisição

solicitada -> autorizada -> em_uso -> concluida; também negada e cancelada. Um veículo só aceita nova requisição quando a anterior estiver concluída, negada ou cancelada (índice único parcial no banco).

Matriz resumida de permissões

Ação	Solicitante	Autorizador	Admin	Motorista	Vigilante
Criar requisição	Sim	Sim	Sim	Não	Sim
Ver requisições	Só as próprias	Todas	Todas	Só as designadas	Próprias + autorizadas/em uso
Autorizar / negar	Não	Sim (não a própria*)	Sim (não a própria*)	Não	Não
Cancelar	Própria, se solicitada	Solicitada/autorizada	Idem autorizador	Não	Não
Registrar saída/chegada	Não	Sim	Sim	Própria viagem, se recurso ligado	Qualquer autorizada/em uso
Atribuir motorista	Não	Sim	Sim	Não	Não
Usuários, veículos, configurações	Não	Não	Sim	Não	Não
Dashboards / relatórios	Relatórios próprios	Sim	Sim (+ governança)	Não	Não

* Exceto se PERMITIR_AUTOAUTORIZACAO estiver ligada. Matriz de apoio: a fonte são as seções seguintes; confira-as em caso de dúvida.

2. Perfil Solicitante

Objetivo

Pedir o uso de um veículo e acompanhar o andamento do próprio pedido. A decisão e o registro de saída e chegada ficam com outros perfis (`backend/app/services/perfis.py:10`).

Pode

- Criar requisição com `nome_solicitante`, `unidade`, `hora_saida` (prevista), `trajeto`, `veiculo_id` e `observacao` opcional (`backend/app/services/validacao.py:13-21,95-111`; `backend/app/rotas/api.py:132-137`).
- Criar apenas se o veículo existir, estiver ativo e livre; caso contrário recebe 409 `veiculo_ocupado` (`backend/app/services/requisicoes.py:548-553`).
- Listar e abrir as próprias requisições (`backend/app/services/requisicoes.py:475,490`).
- Cancelar a própria requisição, e só enquanto `solicitada` (`backend/app/services/requisicoes.py:297-301`).
- Ver a lista de veículos com a disponibilidade calculada no servidor (`backend/app/services/veiculos.py:14-31`).

Não pode

- Autorizar ou negar (`backend/app/rotas/api.py:152-163`), nem registrar saída ou chegada (`backend/app/rotas/api.py:166-177`).
- Atribuir motorista (`backend/app/rotas/api.py:180-185`).
- Registrar ou enviar fotos de checklist (`backend/app/rotas/api.py:239-261`). Ler o checklist: o dono recebe 403 e o solicitante alheio recebe 404 (`backend/app/services/checklist.py:7-12`).
- Acessar dashboards: recebe 403 (`backend/app/services/dashboards.py:172`).
- Cancelar requisição já `autorizada` (`backend/app/services/requisicoes.py:301`).

Visibilidade de dados

- `usuario_id` vem sempre do JWT, nunca do corpo (`backend/app/services/requisicoes.py:554`). Requisição alheia responde 404, sem revelar que existe (`backend/app/services/requisicoes.py:475-476`).
- Os ids de quem autorizou, registrou ou cancelou só aparecem se o autor for o próprio dono (`backend/app/services/requisicoes.py:441-445`). O histórico não traz o campo `por` (`backend/app/services/requisicoes.py:140,412`). Na linha do tempo a equipe aparece só como "Autorizador" (`backend/app/services/requisicoes.py:157,259-261`).
- Nomes e fotos da equipe nem são consultados (`backend/app/services/requisicoes.py:382-387`).
- Relatórios: o escopo é sempre o id do JWT (`backend/app/services/relatorios.py:814`).

Estados/transições que afeta

- Cria a requisição em `solicitada` e pode levá-la a `cancelada` (`backend/app/services/requisicoes.py:82-87`).
- `solicitada` ocupa o veículo; `cancelada` o libera (`supabase/migrations/20261107120300_veiculo_ocupado.sql:12-14`).
- `cancelada` é terminal, sem DELETE físico (`supabase/migrations/20261117120200_vigilante_requisicoes.sql:77-83`).

Recursos opcionais que alteram o perfil

- Quilometragem: `km_saida` e `km_chegada` ficam sempre visíveis, mesmo com o recurso desligado (`backend/app/services/requisicoes.py:404-407`). O solicitante nunca informa km (`backend/app/services/requisicoes.py:338,368-372`).
- Motorista: com o recurso ligado, o solicitante vê id e nome do motorista da viagem (`backend/app/services/requisicoes.py:389-391,410`).
- Checklist: ligado ou não, o solicitante não tem acesso (ver "Não pode").

Garantias no banco (trigger/RLS/constraint)

- Índice único parcial `ux_requisicoes_veiculo_aberta`: uma requisição aberta por veículo, segura contra concorrência (`supabase/migrations/20261107120300_veiculo_ocupado.sql:12-14`). O erro 23505 vira `veiculo_ocupado` (`backend/app/repositorios/supabase_repo.py:207`).

- Trigger `requisicoes_validar`: nasce `solicitada`, com solicitante e veículo ativos, e carimbos definidos pelo servidor (`supabase/migrations/20261117120200_vigilante_requisicoes.sql:47-66`). Cancelar exige ser admin/autorizador ou o dono com status `solicitada` (`:166-170`).
- Mudar de status exige um ator ativo (`supabase/migrations/20261117120200_vigilante_requisicoes.sql:148-158`).
- RLS ativo e sem privilégios para `anon` e `authenticated`; a leitura por papel foi removida de propósito (`supabase/migrations/20261107121300_sem_select_direto_e_ator_auditoria.sql:8-19,60-72`). A proteção contra IDOR fica na camada de serviço.

Pontos em aberto/divergências do CLAUDE.md

- `nome_solicitante` e `unidade` vêm do corpo e não são cruzados com `perfis` (`backend/app/services/validacao.py:101-110`). O CLAUDE.md só fixa `usuario_logado` pelo JWT.
- O solicitante não cancela `autorizada`. Isso está de acordo com o CLAUDE.md, mas uma requisição parada trava o veículo, que não tem prazo de uso.
- O nome do motorista é exposto ao solicitante, enquanto a equipe é ocultada (`backend/app/services/requisicoes.py:410` e `:261`). Falta decisão de LGPD.
- `perfis.py:10` não lista `cancelar`, embora o serviço o permita.
- Não verifiquei a cobertura de testes específica do solicitante.

3. Perfil Autorizador

Objetivo

Decidir sobre as requisições de veículo (autorizar ou negar) e registrar a saída e a chegada reais. O papel vem de `perfis.role`, nunca do cliente (`backend/app/auth/decoradores.py:80-88`). A equipe (`STAFF`) é formada por autorizador e admin (`backend/app/services/requisicoes.py:30`).

Pode

- Autorizar ou negar requisições `solicitada` (`requisicoes.py:57-69`; rotas em `backend/app/rotas/api.py:152-163`).
- Registrar saída de requisição `autorizada` e chegada de requisição `em_uso` (`requisicoes.py:70-81`; `api.py:166-177`).
- Atribuir, trocar ou remover o motorista em requisições `autorizada` ou `em_uso`, com o recurso Motorista ligado (`requisicoes.py:38`, `:317-319`, `:519-541`).
- Ao autorizar sem escolher motorista, o sistema designa o único motorista ativo, se houver exatamente um (`requisicoes.py:626-639`).
- Cancelar requisição `solicitada` ou `autorizada`, de qualquer dono (`requisicoes.py:82-87`, `:297-299`).
- Criar requisições como qualquer usuário (`api.py:132-137`).
- Registrar e ver checklist com fotos, quando o recurso está ligado (`backend/app/services/checklist.py:226`; `api.py:239-269`).
- Consultar dashboards (sem o painel de governança) e relatórios conforme o papel (`backend/app/services/dashboards.py:143-157`; `backend/app/services/relatorios.py:131`).

Não pode

- Autorizar ou negar a própria requisição, salvo com `PERMITIR_AUTOAUTORIZACAO=true`, desligada por padrão (`requisicoes.py:304-305`, `:687-693`; `backend/app/config.py:105`).
- Gerenciar usuários, grupos, veículos ou Configurações: são rotas só de admin (`backend/app/rotas/admin.py:33`).
- Ver o painel de governança dos dashboards (`dashboards.py:153`).
- Coletar localização: latitude e longitude enviadas por ele são ignoradas (`requisicoes.py:337`, `:576`).

Visibilidade de dados

- Vê todas as requisições, com nome e foto de quem solicitou, autorizou e registrou (`requisicoes.py:379`, `:417-423`, `:490`).
- Vê o histórico completo, com o campo `por` (`requisicoes.py:412`).
- Vê o checklist e as fotos (URL assinada) de qualquer requisição (`checklist.py:226`).
- Dos motoristas, vê só id e nome (`requisicoes.py:409-410`, `:428-431`).

Estados/transições que afeta

- `solicitada -> autorizada` ou `negada`; `autorizada -> em_uso`; `em_uso -> concluida`; `solicitada`/`autorizada -> cancelada` (`requisicoes.py:57-88`).
- A transição é condicional ao estado de origem; se outra pessoa mudou a requisição, devolve erro de conflito (`requisicoes.py:579-581`).
- Cada transição dispara notificação após o commit (`requisicoes.py:582`).

Recursos opcionais que alteram o perfil

- **MFA:** com o recurso ligado, exige sessão aal2 com TOTP; sem ela a API responde `mfa_requerido` (`decoradores.py:33-41`, `:90-97`).
- **Quilometragem:** km de saída e de chegada passam a ser obrigatórios. O km de chegada não pode ser menor que o de saída, e km já gravado é imutável (`requisicoes.py:367-372`, `:656-685`).
- **Checklist:** o checklist do momento é exigido antes da saída e da chegada (`requisicoes.py:641-654`).
- **Motorista:** habilita a atribuição e o campo opcional `motorista_id` ao autorizar (`requisicoes.py:334-335`, `:613-634`). Com o recurso desligado, a API recusa a atribuição (`requisicoes.py:479-481`).

Garantias no banco (trigger/RLS/constraint)

- O trigger `requisicoes_validar` impõe a máquina de estados, exige responsável ativo com role adequado e carimba os horários no servidor (`supabase/migrations/20261117120200_vigilante_requisicoes.sql:138-217`).
- Dados de autorização, saída, chegada e cancelamento já gravados são imutáveis (mesmo arquivo, `:120-124`).
- Km gravado é imutável e só entra junto da transição (mesmo arquivo, `:244-258`).
- RLS ativo em todas as tabelas; a equipe lê todas as requisições, o solicitante só as próprias (`20261107120800_rls_por_papel_e_seed_admin.sql:70-76`, `:90-95`).
- O checklist só é aceito de ator ativo com role permitido (`20261118120000_vigilante_checklist.sql:97`, `:113-114`).

Pontos em aberto/divergências do CLAUDE.md

- **Autoautorização:** só é barrada no Flask. Nenhum trigger compara quem autoriza com o dono da requisição, e a migration `20261107121400_correcoes_revisao_4.sql:73` a deixou como "PENDENTE". Com a flag ligada, a exceção não é marcada na auditoria.
- **Impasse operacional:** um autorizador sem par não autoriza as próprias requisições; só o admin destrava (`requisicoes.py:304-305`).
- **Cancelamento pela equipe:** o CLAUDE.md só cita o cancelamento pelo solicitante (`requisicoes.py:297-299`; trigger `:166-170`).
- **Localização:** a equipe registra saída e chegada sem coordenadas, mesmo com a coleta ligada; a exigência do banco vale só para o role motorista (`requisicoes.py:576`; `20261117120200_vigilante_requisicoes.sql:190-196`).
- **Permissões em `/api/eu`:** `atribuir_motorista` não consta na lista do autorizador (`backend/app/services/perfis.py:11-19`).

4. Perfil Admin

Objetivo

Administrar contas, convites, veículos e recursos opcionais do sistema. A role vem de `perfis` no banco, nunca do cliente (`backend/app/auth/decoradores.py:84-89`). Toda rota `/api/admin` exige role `admin` (`backend/app/rotas/admin.py:29-33`), e cada service reconfere com `exigir_admin` (`backend/app/services/admin_usuarios.py:70-73`).

Pode

- Convidar usuários por e-mail `@*.agu.gov.br` ou Gmail, com validade de 1 a 30 dias (padrão 7) e role inicial à escolha (`backend/app/services/validacao_admin.py:14,18,90-105`).
- Reenviar ou revogar convites abertos ou expirados (`backend/app/services/convites.py:130,255-300`).
- Editar nome, unidade e role de usuários; ativar e desativar contas (`backend/app/services/admin_usuarios.py:255-344`).
- Trocar ou remover a foto de perfil de outra conta (`backend/app/rotas/admin.py:139-159`).
- Criar e editar veículos (descrição e placa) e ativar ou desativar (`backend/app/services/admin_veiculos.py:51-83`).
- Ligar e desligar os 11 recursos opcionais (`backend/app/services/admin_configuracoes.py:19-81,145-160`).
- Consultar o histórico de logins e as coordenadas de saída e chegada do motorista, ambos com consulta auditada (`backend/app/rotas/admin.py:227-239`).
- Configurar a IA opcional (`backend/app/rotas/admin.py:268-285`).
- Atuar como autorizador: autorizar, negar, registrar saída e chegada, cancelar (`backend/app/services/perfis.py:20-30`).

Não pode

- Alterar o próprio papel nem se desativar (`backend/app/services/admin_usuarios.py:65-66,261-262,283-284`).
- Rebaixar ou desativar o último admin ativo (`backend/app/services/admin_usuarios.py:251-253`).
- Apagar usuários, veículos ou configurações, pois não há DELETE físico (`backend/app/services/admin_veiculos.py:1`).
- Alterar e-mail ou id da conta, nem enviar campos fora da lista fechada (`backend/app/services/validacao_admin.py:34-40,108-120`).
- Trocar a foto de conta inativa (`backend/app/services/admin_usuarios.py:314-315`).
- Ligar o MFA sem estar em sessão com segundo fator (`backend/app/services/admin_configuracoes.py:155-156`).
- Ligar um recurso sem suas dependências, nem desligar um do qual outro depende (`backend/app/services/admin_configuracoes.py:90-99,153`).

Visibilidade de dados

- Vê todos os usuários com e-mail (vindo do Auth) e último acesso (`backend/app/services/admin_usuarios.py:192-199`).
- Vê todos os convites e todos os veículos, incluindo se há requisição aberta (`backend/app/services/admin_veiculos.py:24-34`).
- É o único perfil que vê coordenadas do motorista, e a leitura é auditada (`backend/app/rotas/admin.py:235-239`).
- A trilha de auditoria é acessada pelo relatório restrito a admin (`backend/app/services/relatorios.py:173-177`).
- Vê todas as requisições (`backend/app/services/perfis.py:20-30`).

Estados/transições que afeta

- Conta: ativa e inativa (desativar grava o perfil e depois bloqueia no Auth; ativar desbloqueia antes) (`backend/app/services/admin_usuarios.py:278-295,331-344`).
- Convite: aberto, expirado, usado, revogado; o admin leva aberto ou expirado a revogado ou a um novo convite (`backend/app/services/convites.py:103-108,255-300`).
- Veículo: ativo e inativo, de forma idempotente (`backend/app/services/admin_veiculos.py:69-77`). Desativar não cancela requisição em andamento (`backend/app/services/admin_veiculos.py:29`).

- Requisição: pode autorizar, negar, registrar saída e chegada e cancelar (`backend/app/services/perfis.py:20-30`).

Recursos opcionais que alteram o perfil

- `mfa`: ligado, o login de admin e autorizador exige TOTP (`backend/app/auth/decoradores.py:90-97`).
- `motorista`, `motorista_registra`, `motorista_localizacao` e `vigilante_km` ampliam os papéis que o admin atribui e as coordenadas que ele consulta (`backend/app/services/admin_configuracoes.py:57-80`).
- Desligar um recurso nunca apaga dados já gravados (`backend/app/services/admin_configuracoes.py:4-5`).

Garantias no banco (trigger/RLS/constraint)

- `perfis_guarda_admin`: serializa com advisory lock, exige admin ativo como responsável, recusa auto-alteração (AG008) e a perda do último admin ativo (AG009) (`supabase/migrations/20261107121400_correcoes_revisao_4.sql:139-225`).
- Primeiro admin só pelo seed (AG002) (`supabase/migrations/20261107121400_correcoes_revisao_4.sql:164-166`).
- `perfis` não aceita DELETE nem TRUNCATE (`supabase/migrations/20261107120400_transicoes_e_auditoria.sql:124-130`).
- `configuracoes_validar`: exige admin ativo, impõe as dependências (AG010) e audita 11 chaves (`supabase/migrations/20261118120100_configuracoes_ag010_mensagem.sql:14-84`).
- Tabela `configuracoes` com linha única, sem DELETE nem TRUNCATE, RLS ativo e sem grants para `anon` e `authenticated` (`supabase/migrations/20261107120700_configuracoes.sql:25-27,43-50,109-111`).

Pontos em aberto/divergências do CLAUDE.md

- A lista de roles fica fixa no cliente, contra a regra de que o front só exibe o que a API informa (`frontend/src/components/ui/chip.tsx:46`, `frontend/src/app/admin/usuarios/novo/page.tsx:101`).
- Convidar ou alterar para `motorista` é aceito mesmo com o recurso `motorista` desligado (`backend/app/services/convites.py:236-253`, `backend/app/services/admin_usuarios.py:255-276`).
- Desativar usuário não trata requisições abertas nem viagens atribuídas a ele (`backend/app/services/admin_usuarios.py:278-295`).
- Ligar o MFA confere só o ator, e outros admins e autorizadores sem TOTP ficam bloqueados (`backend/app/services/admin_configuracoes.py:155`, `backend/app/auth/decoradores.py:93`).
- A auditoria da troca de nome é uma segunda chamada, não atômica com o UPDATE (`backend/app/services/admin_usuarios.py:265-275`).
- `ROLES` e `RECURSOS` repetem o `CATALOGO` e a lista de chaves do trigger (`backend/app/services/validacao_admin.py:12,19`).

5. Perfil Motorista

Objetivo

Permitir que o condutor designado acompanhe as próprias viagens e, se o admin habilitar, registre a saída e a chegada. É um recurso opcional, desligado por padrão (`backend/app/services/motorista.py:1-7`; `supabase/migrations/20261111120100_motorista_recurso.sql:65-72`).

Pode

- Listar e abrir somente as viagens em que `motorista_id` é o seu id, vindo do JWT (`backend/app/services/requisicoes.py:466-471`, `493-496`; `backend/app/rotas/api.py:194-209`).
- Ver a linha do tempo completa da própria viagem (`backend/app/services/requisicoes.py:413-415`).
- Registrar saída (`autorizada->em_uso`) e chegada (`em_uso->concluida`) da viagem designada a ele, com `motorista` e `motorista_registra` ligados (`backend/app/services/requisicoes.py:289-296`; `backend/app/rotas/api.py:166-177`).
- Registrar e ler o checklist da própria viagem. A escrita exige `motorista_registra` (`backend/app/services/checklist.py:220-234`).
- Receber notificações de designação, remoção e cancelamento da viagem (`backend/app/services/notificacoes.py:85-95`).

Não pode

- Criar requisição (`backend/app/services/requisicoes.py:545-546`).
- Autorizar, negar ou cancelar, na API e no banco (`backend/app/services/requisicoes.py:289-296`; `supabase/migrations/20261112120200_motorista_registra_transicao.sql:178-183`).
- Designar ou trocar motorista (`backend/app/rotas/api.py:180-185`) nem listar motoristas (`api.py:188-191`).
- Ver dashboards (`backend/app/rotas/dashboards.py:40-56`) ou relatórios (`backend/app/services/relatorios.py:131`, `618-619`).
- Registrar a viagem de outro motorista (AG035) (`supabase/migrations/20261112120200_motorista_registra_transicao.sql:192-195`).

Visibilidade de dados

- A viagem de outro motorista responde 404, sem revelar que existe (`backend/app/services/requisicoes.py:469-470`).
- Os nomes da equipe saem como "Equipe". O motorista designado aparece como "Motorista" (`backend/app/services/requisicoes.py:254-261`).
- Vê os dados da requisição, incluindo `nome_solicitante`, `unidade`, `trajeto` e `observacao` (`frontend/src/app/motorista/detalhe/page.tsx:42-47`). A `observacao` só é retirada do vigilante (`backend/app/services/requisicoes.py:446-447`).
- As coordenadas nunca chegam a ele: só o admin as lê, e cada leitura é auditada (`backend/app/services/localizacao.py:3-8`; `supabase/migrations/20261112120100_localizacoes_viagem.sql:4-8`).
- Os botões vêm de `acoes_permitidas` e `campos_acao`, sem regra de role no cliente (`frontend/src/app/motorista/detalhe/page.tsx:58-63`).

Estados/transições que afeta

- `autorizada->em_uso` grava `saida_registrada_por` e `saida_real_em`. `em_uso->concluida` grava `chegada_registrada_por` e `chegada_real_em` (`supabase/migrations/20261112120200_motorista_registra_transicao.sql:210-217`, `305-319`).
- A designação só existe a partir de `autorizada` (CHECK) e pode ser trocada em `autorizada` e `em_uso` (`supabase/migrations/20261111120100_motorista_recurso.sql:175-178`, `218-221`; `backend/app/services/requisicoes.py:38`).
- Com um único motorista ativo, ele é designado por padrão ao autorizar (`backend/app/services/requisicoes.py:626-639`).

Recursos opcionais que alteram o perfil

- `motorista`: sem ele, a API recusa o acesso do motorista com 404 `recurso_desligado` e ignora `motorista_id` (`backend/app/services/requisicoes.py:479-481`, `621-622`).

- `motorista_registra`: exige `motorista` (CHECK/AG010) e libera saída e chegada (`supabase/migrations/20261111120100_motorista_recurso.sql:74-78`, `111-114`).
- `motorista_localizacao`: com a flag ligada, latitude e longitude são obrigatórias na saída e na chegada (AG036) (`backend/app/services/requisicoes.py:595`; `supabase/migrations/20261112120200_motorista_registra_transicao.sql:286-296`).
- `quilometragem` e `checklist` valem também para ele (`backend/app/services/requisicoes.py:367-372`, `641-654`).
- Desligar um recurso não apaga dados já gravados (`backend/app/services/requisicoes.py:404-405`).

Garantias no banco (trigger/RLS/constraint)

- O ramo `motorista` de `requisicoes_validar` exige as flags (AG034), o designado do registro antigo (AG035) e as coordenadas (AG036) (`supabase/migrations/20261112120200_motorista_registra_transicao.sql:178-204`).
- A RPC `registrar_transicao_motorista` é `SECURITY DEFINER`, usa `FOR UPDATE` e só o `service_role` a executa. A localização e a transição vão na mesma transação (`20261112120200_motorista_registra_transicao.sql:228-329`).
- O trigger `trg_requisicoes_validar_motorista` valida ator, existência, atividade e role, e audita `motorista_designado` (`supabase/migrations/20261111120100_motorista_recurso.sql:186-260`).
- `localizacoes_viagem` é append-only e tem RLS `using(false)`. O `service_role` só tem SELECT e a escrita é só pela RPC (`supabase/migrations/20261112120100_localizacoes_viagem.sql:66-76`, `123-127`, `201-208`).
- A FK `motorista_id` é `ON DELETE RESTRICT`, e não há DELETE físico (`supabase/migrations/20261111120100_motorista_recurso.sql:161-164`).

Pontos em aberto/divergências do CLAUDE.md

- O CLAUDE.md não menciona o perfil `motorista` (nem `vigilante`), a localização, `motorista_registra` nem `motorista_id`. Estão só em `docs/plano-motorista-vigilante.md`, e o CLAUDE.md prevalece em caso de conflito.
- Um motorista desativado ou rebaixado continua designado. A própria migration reconhece o efeito e o backend não o trata (`supabase/migrations/20261111120100_motorista_recurso.sql:52-54`).
- Com o recurso desligado, o motorista perde o acesso ao histórico, embora os dados permaneçam. Isso não está registrado como decisão (`backend/app/services/requisicoes.py:468`).
- A `observacao` e o `usuario_id` do solicitante chegam ao motorista, ao contrário do vigilante (`backend/app/services/requisicoes.py:439-448`), o que merece decisão de minimização (LGPD).
- A tela inicial leva todo usuário a `/requisicoes`, não a `/motorista` (`frontend/src/app/page.tsx:12`).
- As migrations do motorista ainda não foram aplicadas, e o fluxo real não tem teste ponta a ponta (E2E com mock: `frontend/e2e/mock-motorista.ts`).

6. Perfil Vigilante

Objetivo

Segurança patrimonial: registrar a saída e a chegada reais dos veículos, sem participar da decisão de autorização (`docs/plano-motorista-vigilante.md` §2, item 3; `backend/app/services/requisicoes.py:285-288`).

Pode

- Criar requisição, como o solicitante (`backend/app/rotas/api.py:133`).
- Registrar saída e chegada de qualquer requisição `autorizada` ou `em_uso`, sempre ativo, sem flag e sem ser o motorista designado (`requisicoes.py:285-288`; `api.py:167,174`; `supabase/migrations/20261117120200_vigilante_requisicoes.sql:198-204`).
- Registrar o checklist de saída e de chegada, com fotos, quando o recurso estiver ligado (`backend/app/services/checklist.py:228`; `api.py:240,256`; `20261118120000_vigilante_checklist.sql:111`).
- Informar km de saída e de chegada, só se a API pedir (`requisicoes.py:369-371`).

Não pode

- Autorizar ou negar (`api.py:153,160`; `20261117120200:198-204`).
- Cancelar, nem a própria requisição (`requisicoes.py:285-288`; `20261117120200:162-164`).
- Atribuir motorista (`api.py:181`).
- Ver dashboards (`backend/app/services/dashboards.py:147`) ou relatórios (`backend/app/services/relatorios.py:131`).
- Ler o checklist já registrado (`checklist.py:228`).

Visibilidade de dados

- Lista e consulta: as próprias requisições mais as `autorizada` e `em_uso` de qualquer pessoa (`requisicoes.py:472-473,492`).
- Nas requisições de outros, a `observacao` é removida (`requisicoes.py:446-447`); ids e nomes de quem agiu só aparecem se o ator for o dono (`requisicoes.py:441-445`; linha do tempo sem nomes, `:413-415`).
- Nome do solicitante, trajeto e unidade das requisições de outros continuam visíveis (só `observacao` é removida, `:446-447`).
- Requisição alheia em outro estado retorna 404 (`requisicoes.py:474-476`).

Estados/transições que afeta

- `autorizada -> em_uso` (saída) e `em_uso -> concluida` (chegada). Os carimbos `saida_real_em` e `chegada_real_em` são do servidor, e `saida_registrada_por` e `chegada_registrada_por` guardam o vigilante (`20261117120200:198-217`).
- Qualquer outra transição é recusada com AG002 (`20261117120200:201-204`).

Recursos opcionais que alteram o perfil

- Quilometragem e `vigilante_km` (desligada por padrão): `vigilante_km` exige `quilometragem` (CHECK em `20261117120100_vigilante_km_flag.sql:25-28`; `backend/app/modelos.py:43`). Ligadas as duas, o vigilante informa km de saída e de chegada (`requisicoes.py:371`).
- Quilometragem ligada e `vigilante_km` desligada: o vigilante registra sem km e o km fica nulo (`20261117120200:260-277`). A API ignora o corpo de km (`requisicoes.py:665-666`).
- Checklist: ligado, é obrigatório antes de saída e chegada (`requisicoes.py:641-654`). Desligar um recurso não apaga dados.

Garantias no banco (trigger/RLS/constraint)

- Trigger `requisicoes_validar`: ator ativo e com papel válido (`20261117120200:155-158`), ramo do vigilante (`:198-204`), responsáveis gravados uma vez (`:117-124`), estados terminais congelados (`:77-83`).
- Trigger `requisicoes_validar_km`: km gravado é imutável (`:245-248`); dispensa de km só para o vigilante com `vigilante_km` desligada (`:268`).
- Trigger `checklists_validar` aceita o vigilante (`20261118120000:111-112`).

- `configuracoes_validar` audita `vigilante_km`, e o erro AG010 é amigável (`20261118120100_configuracoes_ag010_mensagem.sql`).
- O vigilante não lê tabelas pela API (RLS deny-by-default; `supabase/tests/test_vigilante.py:497`).

Pontos em aberto/divergências do CLAUDE.md

- O CLAUDE.md não cita os perfis vigilante e motorista, e diz que o autorizador registra saída e chegada.
- O CLAUDE.md exige km obrigatório com a Quilometragem ligada, mas o vigilante com `vigilante_km` desligada registra sem km (`20261117120200:268`).
- MFA vale só para admin e autorizador (`backend/app/auth/decoradores.py:92`), então o vigilante entra só com senha.
- O vigilante pode registrar a saída da própria requisição; autorizar e negar impedem isso (`requisicoes.py:304`), mas registrar não.
- O relatório de km por vigilante casa linhas por `left(id,8)` e tem teto de 10 000 (`relatorios.py:493`; `backend/app/repositorios/supabase_repo.py:139`).
- As migrations 20261117-18 ainda não foram aplicadas (`supabase db push` pendente).